

4. Implementación por módulo

Frontend

- **Autenticación:** página de login; el token JWT se guarda de forma segura y se inyecta por un interceptor de Axios. Rutas protegidas revisan el rol (`admin`, `gestor`, `consultor`).
- **Carga de documentos:** formulario de carga múltiple con arrastrar/soltar; muestra el estado de cada documento (pendiente/procesando/procesado/error) mediante `polling`.
- **Detalle y corrección:** vista del documento con texto extraído, resumen, categoría (editable si la confianza < 70% o por solicitud) y metadatos.
- **Búsqueda:** barra con modo "palabra clave" o "semántico", filtros por categoría y resultados con vista previa y descarga.
- **Chat RAG:** conversación en lenguaje natural con historial en la sesión; cada respuesta muestra las fuentes (documentos) utilizadas.
- **Dashboard:** gráficas con Chart.js (documentos por categoría, procesados por semana, consultas realizadas).

Backend

- **Seguridad:** `security/` implementa JWT (emisión, verificación, renovación), cifrado BCrypt y decoradores de rol (`require_role`).
- **Validación de archivos:** se valida extensión, MIME y magic bytes antes de persistir; control del tamaño máximo de 25 MB.
- **Procesamiento asíncrono:** al cargar un documento se encola la tarea; el estado avanza y la API expone el detalle del procesamiento.
- **Auditoría:** `services/auditoria.py` registra cada acción sensible (login, carga, edición, descarga) en la tabla `auditoria`.

Base de datos y almacenamiento

- `db/schema.sql` crea las tablas, habilita la extensión **pgvector** y crea el índice **HNSW** sobre `procesamiento.embedding`.
- `db/seed.sql` inserta los roles, las categorías (factura, guía, orden de compra, contrato, acta) y un usuario administrador inicial.
- Los archivos originales se guardan en MinIO (modo contenedor) o en `STORAGE_PATH` local.

Procesamiento de documentos

- Pipeline en `ia/`: extracción de texto (nativo u OCR) → normalización → chunking → embeddings → clasificación → metadatos → resumen → persistencia.

- Estados: `pendiente` → `procesando` → `procesado` (o `error` con motivo).

Integración de IA

- `ia/embeddings.py` usa la API de OpenAI para generar vectores; `ia/ocr.py` usa PaddleOCR con Tesseract de respaldo.
- `ia/rag.py` recupera fragmentos por similitud coseno sobre pgvector y arma el prompt del LLM para responder citando fuentes.
- Todos los parámetros (modelo, chunk, dimensión, API key) vienen de variables de entorno.

Clasificación, resumen y extracción

- `ia/clasificador.py`: prompting estructurado del LLM que devuelve categoría, confianza y metadatos en JSON.
- `ia/resumen.py`: genera un resumen de hasta 150 palabras del texto extraído.

Búsqueda y consulta documental

- Búsqueda textual (`ILIKE` sobre texto y metadatos) y semántica (`vector` con filtro de categoría) en `services/busqueda.py`.
- Consulta RAG en `services/rag.py` reutilizando `ia/rag.py`.

Gestión de errores y validaciones

- Manejo centralizado de excepciones en FastAPI: 400 (validación), 401/403 (seguridad), 404 (inexistente), 422 (esquema), 500 (interno) con mensajes en español.

Seguridad de credenciales y variables de entorno

- `.env` excluido del repositorio mediante `.gitignore`.
- `env.example` documenta las variables requeridas (incluye `OPENAI_API_KEY`, `JWT_SECRET`, `DATABASE_URL`). Nunca se incluyen valores reales.